

ADR 0003 — Segurança e LGPD do backend

- **Status:** Aceito
- **Data:** 2026-06-23
- **Decisor:** redteam (com `arq`, `back`)
- **Relacionado:** ADR 0001 · ADR 0002
- **Escopo:** threat modeling defensivo/autorizado.

1. Onde mora a autorização (por arquitetura)

	(A) SPA + Supabase (escolhida)	(B) SPA + backend dedicado
Fronteira de segurança	RLS no Postgres (chave publishable é pública por design)	Middleware de authz no servidor (banco não exposto)
Modo de falha	RLS esquecida numa tabela nova → tudo vaza, sem erro visível	Middleware esquecido num endpoint → endpoint exposto, testável
Defesa em profundidade	RLS + Edge Function (<code>service_role</code>) = 2 camadas	Backend + regra de serviço = 1 camada clara
Custo do erro humano	Um <code>using (true)</code> numa policy = todos os dados	Omissão de um middleware = um endpoint

Veredito: (A) é adequada para o estágio **se** os P0 abaixo forem implementados antes do primeiro usuário real. RLS incompleta em produção é pior que qualquer outra decisão de arquitetura.

2. Threat model das funcionalidades críticas (STRIDE enxuto)

Funcionalidade	Ameaça	Mitigação
Reserva	Race / reserva dupla (TOCTOU)	Índice único parcial + <code>SELECT FOR UPDATE</code> na RPC
	Cancelar reserva alheia (IDOR)	RLS/authz: só a própria reserva; auth de convidado quando houver
	Marcar <code>status</code> direto via PostREST	Escrita de reserva só via Edge Function/RPC (<code>service_role</code>); RLS bloqueia UPDATE de <code>status</code> pelo cliente
Link de compartilhamento	Token adivinhável / enumerável	<code>crypto.getRandomValues(16)</code> = 128 bits; nunca token semântico (o seed de mock <code>festa-rodriigo-25</code> não pode ir p/ produção)
	Varredura de tokens (DoS/custo)	Rate limit por IP no <code>GET /registry/:token</code>
	Token redundante no corpo da resposta	Não devolver <code>listToken</code> no payload (já está na URL)
RSVP / PII	E-mails de convidados expostos	RLS: só o host do evento lê <code>guests</code> ; nunca via anon key
	Quebra da "surpresa"	Resposta omite quem reservou quando <code>surprise_mode = true</code>
Enriquecimento de link	SSRF (IP interno / <code>169.254.169.254</code>)	Allowlist: só http/https, bloquear IP privado/CGNAT, resolver DNS antes do fetch, sem seguir redirect , timeout 3s
	XSS persistente via <code>og:title</code>	Sanitizar metadados (texto puro) antes de gravar

3. LGPD

Tema	Decisão
Residência de dados	Supabase sa-east-1 (São Paulo) → sem transferência internacional, sem necessidade de SCCs. (Região é irreversível após criar o projeto — escolher São Paulo no setup.)
Dados pessoais	Nome e e-mail de convidados; nome/e-mail do host; mensagens.
Bases legais	Host: execução de contrato. Convidado: consentimento no ato do RSVP (aviso + link p/ política).
Minimização	RSVP coleta só o necessário; não expor e-mail de um convidado a outro.
Retenção / exclusão	<code>DELETE /account</code> para o host; anonimização da reserva/recado do convidado ao cancelar; política de anonimizar evento passado após ~90 dias.
DPA	Assinar o DPA padrão do Supabase no dashboard.

4. Hardening priorizado

P0 — antes de qualquer usuário real

1. Índice único parcial na reserva (race condition) — ver **ADR 0002**.
2. **RLS habilitada e fail-closed em TODAS as tabelas**. Auditar tabela sem policy:

```
``sql select tablename from pg_tables where schemaname = 'public' and not exists (select 1 from pg_policies where tablename = pg_tables.tablename); ``
```

1. Remover `listToken` do corpo de `GET /registry/:token` .

- 2. **Validação anti-SSRF** antes de qualquer enriquecimento de link.
- 3. **Ownership check** em mutações de presente (o gift pertence ao evento do host).

P1 – sprint seguinte

- 1. **Rate limiting por IP** (registry 60/min, RSVP 5/min, recados 10/min, reserva 3/min).
- 2. **Não expor e-mails de convidados** a outros convidados; dashboard com e-mail só p/ host autenticado.
- 3. **Headers de segurança** no Netlify (`netlify.toml`): CSP, `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy` .
- 4. **Sanitização anti-XSS** de todo input renderizado (título, mensagem, nome/descrição de presente, recado).
- 5. **Teste de entropia** do token de share (≥128 bits, só chars base64url).

P2 – antes de escalar

- 1. Mecanismo de **direito de exclusão** (LGPD Art. 18) + retenção automática.
- 2. **CORS** explícito (origem do SPA), nunca `*` em endpoint autenticado.
- 3. Cookie de sessão do host `HttpOnly; Secure; SameSite=Lax` (via `@supabase/ssr`).
- 4. Desabilitar **introspection** do PostgreSQL em produção.
- 5. Confirmar `sa-east-1` na criação do projeto.

5. Testes de segurança a adicionar

Camada	Teste
Unit	<code>generateListToken()</code> ≥ 22 chars, só base64url; <code>validateStoreUrl()</code> rejeita <code>169.254.169.254</code> , <code>file://</code> , IP 10.x
Integração (MSW)	<code>POST /reservation</code> simultâneo → 201 + 409; <code>GET /registry/:token</code> não retorna <code>listToken</code> nem <code>guests[].email</code> ; <code>GET /dashboard</code> sem auth → 401; <code>PATCH /gifts/:id</code> de outro evento → 403
E2E (Playwright)	reserva concorrente mostra "reservado" sem nome; recado com <code><script></code> renderiza como texto
Pentest manual	flood de RSVP → 429; <code>storeLink=http://169.254.169.254/...</code> → rejeitado; <code>GET /rest/v1/guests</code> com anon key → 0 linhas (RLS)

Fontes (2026-06-23)

Supabase RLS · Supabase regiões · ANPD — transferência internacional · CWE-362, CWE-639, CWE-918, CWE-79, CWE-359, CWE-287.